

Academic year
2025 - 2026

Exam Programming: PINN

Using Physics Informed Neural Networks to solve Partial Differential Equations

Des De Borger



University of Antwerp
| Faculty of Science

Contents

1. Introduction	1
2. Theoretical background	3
3. Damped Harmonic Oscillator	5
3.1. Understanding an the PINN example	5
3.1.1. Questions on the method	5
3.1.2. Questions on the code	8
4. Burger's Equation	11
5. Conclusion	13
A. Equation derivations	15
B. Figures	17

1. Introduction

2. Theoretical background

3. Damped Harmonic Oscillator

$$my''(t) + cy'(t) + ky(t) = 0 \quad (3.1)$$

3.1. Understanding an the PINN example

3.1.1. Questions on the method

Question 1

Identify the total loss function in the code and write its expression down in your report. Identify the different contributions to the loss function.

The total loss function has three terms and is displayed below:

$$\mathcal{L}_{tot} = \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics} + \lambda_{i.c.} \mathcal{L}_{i.c.} \quad (3.2)$$

The first term is the mean squared error of the predicted data compared to the 15 noisy observation points:

$$\mathcal{L}_{data} = \frac{1}{M} \sum_{j=1}^M (\hat{y}(t_j) - y_j)^2 \quad (3.3)$$

with $j = 1, 2, \dots, 15$. This term is the total loss function of a standard, non-PI NN and gives the model direct information about the correct solution of the equation with the relevant boundary conditions, but doesn't check for any non-physical behavior.

The second term is the mean squared residual of the ODE (see equation 3.1) at the 200 evenly spaced collocation points:

$$\mathcal{L}_{physics} = \frac{1}{N} \sum_{i=1}^N (m\hat{y}''(t_i) + c\hat{y}'(t_i) + k\hat{y}(t_i))^2 \quad (3.4)$$

with $i = 1, 2, \dots, 200$. This term gives the model information about how much the predicted solution violates the mechanics of the system, without checking if the predicted solution matches the observed data.

The third term is the sum of squared errors of the two initial conditions:

$$\mathcal{L}_{i.c.} = (\hat{y}(0) - y_0)^2 + (\hat{y}'(0) - dy_0)^2 \quad (3.5)$$

This term enforces the initial conditions and applies only to the very first point $t = 0$.

The constants $\lambda_{physics}$ and $\lambda_{i.c.}$ determine the relative weight of every term in the loss function. Because it is important that the initial conditions are met, $\lambda_{i.c.}$ is chosen at a much larger value than $\lambda_{physics}$. This is why in the given program, the values are set at $\lambda_{physics} = 0.01$ and $\lambda_{i.c.} = 10$.

Question 2

Is the inclusion of the initial condition loss essential for this particular example? When would it be essential?

To investigate the effect of the initial condition term, the term can be dropped out of the loss function to compare the results. Figure 3.1 shows the predicted solutions of the ODE if the $\mathcal{L}_{i.c.}$ is included and excluded. In this specific case, there does not seem to be a big difference which indicated that the initial condition loss is not essential. This is because the noisy observations point, which also contribute to the loss function, are close enough to $t = 0$ to give enough information for the model to accurately predict the initial conditions.

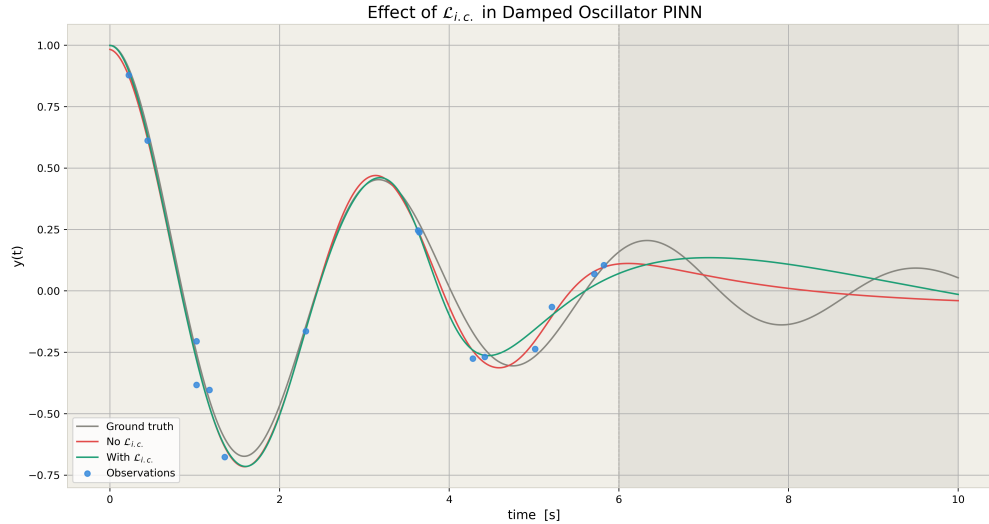


Figure 3.1.: Predicted solutions to the ODE of the $\mathcal{L}_{i.c.}$ is included and excluded. In this specific case, the $\mathcal{L}_{i.c.}$ is not absolutely necessary, because the noisy observation points give enough information for the model to accurately predict the initial conditions.

We do not always have access to observations close to the initial time t_0 . However, if the initial conditions are known and need to be enforced, $\mathcal{L}_{i.c.}$ is a crucial term in the total loss function.

Question 3

The loss function used here is based on the mean squared errors (MSE) between predictions and data points or initial condition, and on the mean squared residual (MSR) of the PDE for the collocation points. Why are these specific functional forms used, and would there be alternative functions that could replace the MSE or MSR?

The mean square error and mean square residuals are the most commonly used loss functions for regression models. Any loss function has to be symmetrical and positive for all possible values, so that errors cannot cancel each other out. Furthermore, it is desired that the loss function is differentiable so that its gradient can be computed.

The MSE and MSR check both of these boxes. However, since the errors are squared before averaging, these functions heavily punish and are sensitive to large outliers. This can lead to problems when very noisy data points are included in the calculation for the loss function. However, this effect is good for the residuals since it is essential that the predicted solution does not break the laws of physics. There are alternative loss functions with different (dis)advantages [1]:

Mean Absolute Error The Mean Absolute Error (MAE) does not average the squares, but the absolute distances:

$$\mathcal{L}_{MAE} = \frac{1}{M} \sum_{j=1}^M |\hat{y}(t_j) - y_j| \quad (3.6)$$

Because the errors are not squared, this loss function is less sensitive to large outliers, which can make it more robust. However, it is not differentiable at zero, which can lead to problems for some optimisation algorithms.

Huber Loss The Huber Loss is a hybrid of the MAE and the MSE which is differentiable, but less sensitive to outliers than the MSE. It is defined as follows:

$$L_{\delta}(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| \leq \delta \\ \delta (|x| - \frac{1}{2}\delta) & \text{if } |x| \geq \delta \end{cases} \quad (3.7)$$

The Huber Loss behaves like the MSE for $|x| < \delta$ and like the MAE for $|x| > \delta$. The hyperparameter δ can be tuned to make the loss function more or less sensitive to outliers.

Question 4

What is the type of neural network used in this example? How many input nodes, output nodes and hidden layers does it have? How many parameters are then being optimized? What type of activation function is used and why? What quantity or quantities are provided at the input, and what quantity or quantities do we extract from the output of the network? Are the dimensions of this Neural Network architecture adequate, too small or too large for this example? How would you determine this?

A dense network is used in this case. We are trying to solve an ODE, which means to solve for a function $y(t)$. This function takes one input and returns one output, which means the network will also have one input and one output. The parameters for the hidden layers can be given as inputs, in the example script there are 4 hidden layers of 32 nodes. For every connection between nodes has a weight, and every node has a bias. From this we can calculate the total parameter count:

$$\begin{aligned} n &= \text{First layer weights} + \text{First layer bias} \\ &\quad + \text{Hidden layer weights} + \text{Hidden layer bias} \\ &\quad + \text{Last layer weights} + \text{Last layer bias} \\ &= 1 \cdot 32 + 32 + 4(32 \cdot 32 + 32) + 1 \cdot 32 + 1 = 3265 \text{ parameters} \end{aligned} \quad (3.8)$$

Question 5

How many input samples are provided to the neural network model during each of its training cycles or epochs? What kind of samples are provided? Does the network receive again for each epoch the same exact samples? Would there be a better way to provide samples to this network for training over the various epochs?

Question 6

The example uses the Adam algorithm as the so-called optimizer. What is the purpose of this optimizer? What is specific about the Adam algorithm, and what are its main hyper-parameters? Are there viable alternatives for the Adam optimizer?

Question 7

The example also uses an additional optimizer: a learning rate scheduler. What does this scheduler do? Is it a necessary ingredient for training a neural network? What are its main hyperparameters? Are there alternatives that can be usefully applied in this example?

Question 8

How many epochs is the network trained for? How would you determine whether you have trained for too little or too many epochs?

Question 9

Would it make sense to have a set of validation points? How would these look like? How would you use them?

3.1.2. Questions on the code

Question 1

Why are these two lines of code good, or bad, for:

```
torch.manual_seed(42)
np.random.seed(42)
```

Question 2

What is the purpose of the `np.ndarray` statements in this function definition:

```
def analytic(t: np.ndarray) -> np.ndarray:
```

Question 3

The data points are created randomly and then sorted before being offered to the neural net, using this statement:

```
t_obs = np.sort(np.random.uniform(0.1, T_TRAIN, N_OBS))
```

Is this good or bad?

Question 4

The collocation points are created uniformly on a fixed grid, using this statement:

```
t_col = np.linspace(0.0, T_EXTRAP, N_COL)
```

Is this good or bad?

Question 5

What is this statement doing? Is it necessary?

```
optimiser.zero_grad()
```

Question 6

What are these statements doing in the training cycle? Is their order important?

```
loss_total.backward()
optimiser.step()
scheduler.step()
```

Question 7

Why do the collocation time input tensor and the initial condition time input tensor require the gradient option, while the input data tensors do not require a gradient?

```
t_obs_t = to_tensor(t_obs)
y_obs_t = to_tensor(y_obs)
t_col_t = to_tensor(t_col, requires_grad=True)
t_ic_t = to_tensor(t_ic, requires_grad=True)
```

Question 8

When presenting arrays of time points to the input of my neural network, I need to convert them into a pytorch tensor, using this helper function:

```
def to_tensor(arr, requires_grad=False):
    return torch.tensor(arr, dtype=torch.float32,
                        requires_grad=requires_grad).unsqueeze(1)
```

What does the `unsqueeze(1)` method of the `torch.tensor` class do, and why do I need to do this?

Question 9

When computing the first and second derivatives of $y(t)$ we use the `torch.autograd` method. Explain why the `grad_outputs` are set to ones and why we need to take the zero-th element `[0]` of this output in the statements below:

```
dy = torch.autograd.grad(
    y_hat, t,
    grad_outputs=torch.ones_like(y_hat),
    create_graph=True)[0]
```


4. Burger's Equation

5. Conclusion

A. Equation derivations

B. Figures

Bibliography

- [1] GeeksforGeeks. Loss function for linear regression in machine learning, 2024. Accessed: 2026-05-06. 6